# GAN - DC

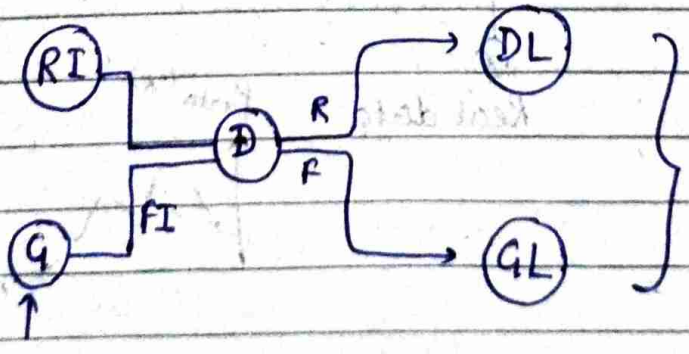
GAN : Generative Adversarial Network
↳ It is a type of deep learning model
↳ Introduced by Ian Goodfellow (2014)
↳ Used for generating new, realistic - looking data by learning the distribution of real data

Two neural networks playing game : (Minimax-Game)

1. Generator: (G)
   ↳ Take random noise z (from distribution like Gaussian or Uniform)
   ↳ Generates fake data that looks like real data

2. Discriminator: (D)
   ↳ Takes input data and decides whether its real (from the dataset) or fake (from the Generator)

★ Generator (G) tries to fool the Discriminator by generating fake data that looks real and the discriminator tries to catch the generator by classifying real v/s fake



## ⊛ MATHS BEHIND GANS:

(G) Generator ↙

$P(X, Y)$
$= P(X|Y)\,P(Y)$
$= P(Y|X)\,P(X)$

↓

Model uses Bayes theorm
if it has to predict something.

$$P(Y|X) = \frac{P(Y \cap X)}{P(X)}$$

↓

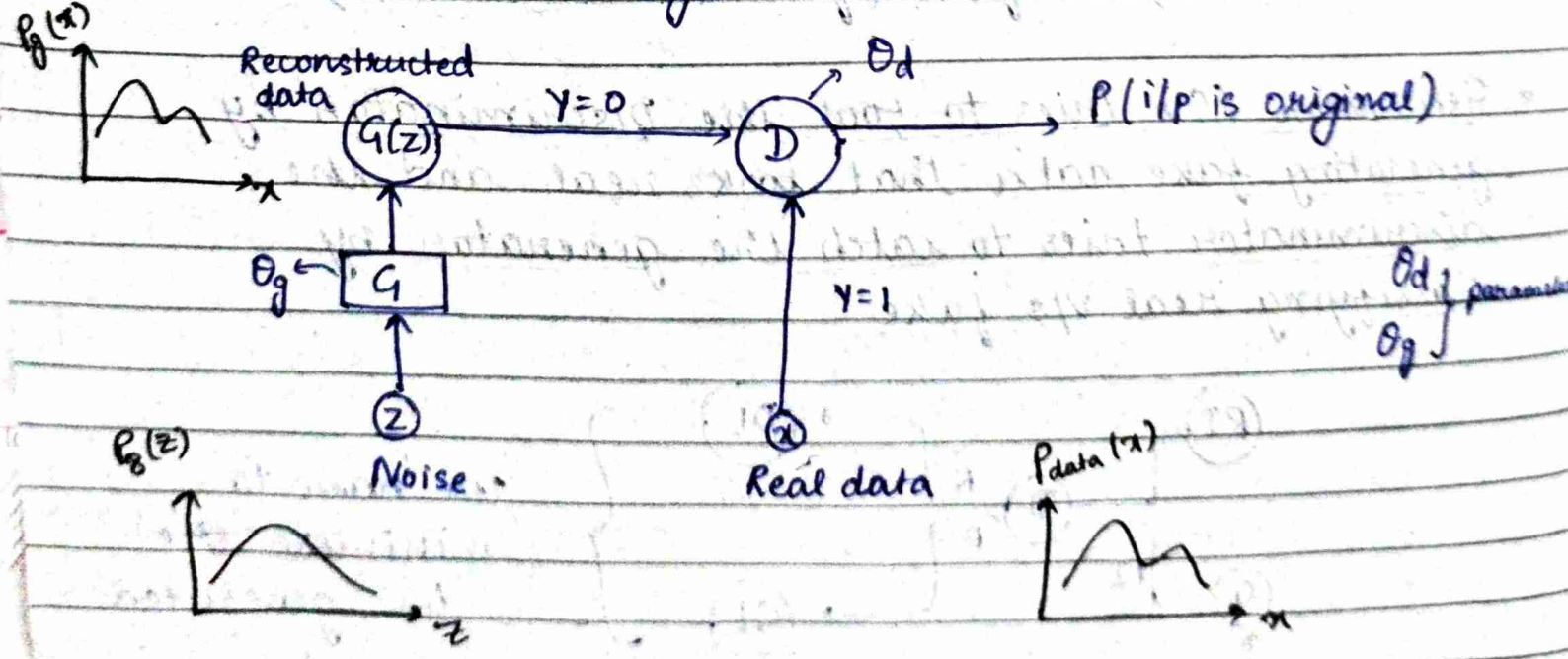
Common model : Naive-Bayes Model

(D) Discriminator ↓

$P(Y|X) = \hat{Y}$

eg :- Logistic regression
Linear regression

Universal Appx Theorm
Neural Networks can appx any function



## Value Function:

$$\min_{G}\max_{D} V(G,D) = E_{x\sim p_{data}}[\ln(D(x))] + E_{z\sim p_z}[\ln(1-D(G(z)))]$$

## Binary Cross Entropy Function:

$$\mathcal{L} = -\sum y \ln \hat{y} + (1-y)\ln(1-\hat{y})$$

[ $\hat{y}$ → predicted
[ $y$ → ground truth level

① when $y=1$, $\hat{y} = D(x) \Rightarrow \mathcal{L} = \ln[D(x)]$ → Real

② when $y=0$, $\hat{y} = D(G(z)) \Rightarrow \mathcal{L} = \ln[1-D(G(z))]$ → Fake

① + ②

Adding, $\mathcal{L} = \ln[D(x)] + \ln[1-D(G(z))]$

Only $E$ missing

↓

Expectation

↳ perform many times and take avg

$\therefore E(\mathcal{L}) = E(\ln(D(x))) + E(\ln(1-D(G(z))))$

$= \sum p_{data}(x) \ln(D(x)) + \sum p_z(z) \ln(1-D(G(z)))$ [Discrete]

↓ continuous → change with $\Sigma \Rightarrow \int$

$= \int p_{data}(x) \ln(D(x)) + \int p_z(z) \ln(1-D(G(z)))$

$= E_{x\sim p_{data}}[\ln(D(x))] + E_{x\sim p_z}[\ln(1-D(G(z)))]$

* Optimise loss function using stochastic process :-

Training loop:

* fix the learning of G *

Inner loop for D:

: take m data samples & m fake data samples

: Update $\theta_d$ by gradient ascent.

$$\frac{\partial}{\partial \theta_d} \frac{1}{m} \left[ \ln[D(x)] + \ln[1 - D(G(z))] \right]$$

* fix the learning of D *

Take m fake data samples

Update $\theta_g$ by gradient descent.

$$\frac{\partial}{\partial \theta_g} \frac{1}{m} \left[ \ln[1 - D(G(z))] \right]$$

What guarantee that our generator will surely replicate

$P_g$ → converges to → $P_{data}$

if it finds Global Minimum of value function

★ For fixed G,

$$V(G,D) = \int_x p_{data}(x) \ln[D(x)] + p_g(x) \ln[1-D(x)]\, dx$$

⇒ Differentiate:

will be maximum for $D(x) \Rightarrow$

$$\frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

★ When fixed $D(x)$

$$\min_G V = E_{x \sim p_{data}} \ln\left[\frac{p_{data}(x)}{p_{data}(x) + p_g(x)}\right] + E_{x \sim p_g}\left[1 - \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}\right]$$

$$= E_{x \sim p_{data}} \ln\left[\frac{p_{data}(x)}{p_{data}(x) + p_g(x)}\right] + E_{x \sim p_g}\left[\frac{p_g(x)}{p_{data}(x) + p_g(x)}\right]$$

**JS divergence** (Two measure similarity b/w probability).

$$JS(P_1 \| P_2) = \frac{1}{2} E_{x \sim P_1} \ln\left[\frac{P_1}{\left(\frac{P_1 + P_2}{2}\right)}\right] + \frac{1}{2} E_{x \sim P_2} \ln\left[\frac{P_2}{\left(\frac{P_1 + P_2}{2}\right)}\right]$$

Multiply and divide by 2

$$\min_G V = E_{x \sim P_{data}} \ln\left[\frac{P_{data}}{\left(\frac{P_{data} + P_g}{2}\right)}\right] + E_{x \sim P_g} \ln\left[\frac{P_g}{\left(\frac{P_g + P_{data}}{2}\right)}\right] - 2\ln 2$$

$$= 2 \cdot JS$$

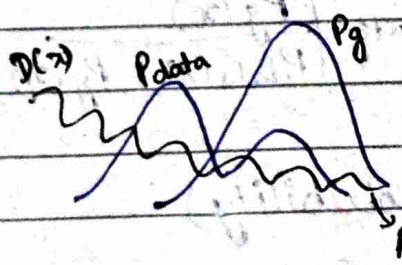
$$\min_G V = 2\, JS(P_{data} \,||\, P_g) - 2\ln 2$$

JS = 0 when

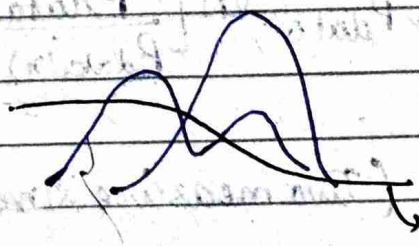
$P_{data} = P_g$

## Graphical Representation :-

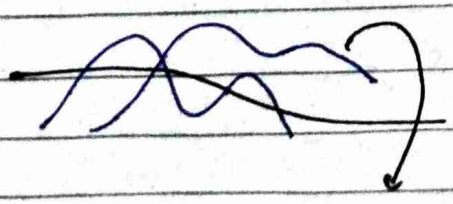
At the beginning



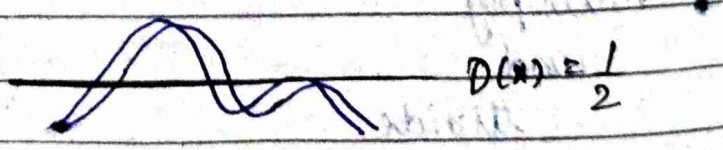

After updating $O_d$




After updating $O_g$




Finally




$P_{data} = P_g$